

# agy CLI vs Antigravity IDE 프롬프팅 차이 가이드

agy CLI (터미널 환경)와 Antigravity / Antigravity IDE (IDE/편집기 내장 환경)는 AI가 자동으로 인지하는 컨텍스트(맥락)의 범위와 상호작용 방식에서 프롬프팅 작성 시 중요한 차이가 있습니다. 각 환경에 맞는 효과적인 프롬프팅 방법을 이해하면 작업 효율을 극대화할 수 있습니다.

## 1. 자동 전달되는 컨텍스트(맥락)의 차이

### agy CLI (터미널 환경)

- 주요 특징:** 터미널 실행 경로(Cwd)와 사용자가 명령어로 명시한 대상 위주로만 맥락을 인지합니다.
- 인지 범위:** 실행한 디렉토리의 파일 구조, 직접 명령어로 전달한 파일의 내용, 터미널 표준 입출력(stdin/stdout).
- 제한 사항:** 사용자가 현재 어떤 코드를 화면에 띄워두고 보고 있는지, 마우스 커서가 어디에 위치해 있는지 알 수 없습니다.

### Antigravity / Antigravity IDE (IDE 환경)

- 주요 특징:** 개발자의 실시간 편집 행위와 에디터 상태가 AI에게 실시간 메타데이터로 자동 연동됩니다.
- 인지 범위:**
  - 활성화된 파일(Active Tab):** 현재 화면에 열려 있는 파일의 경로와 전체 코드.
  - 커서 위치(Cursor Position):** 마우스 커서가 올라가 있는 라인 및 열 위치.
  - 선택 영역(Selection):** 마우스 드래그로 지정한 코드 블록.
  - 기타 맥락:** 열려 있는 다른 탭 목록, 최근 에러 로그 등.

## 2. 환경별 효과적인 프롬프팅 방식 비교

| 비교 항목       | agy CLI (터미널)                                             | Antigravity IDE (IDE 내장)                      |
|-------------|-----------------------------------------------------------|-----------------------------------------------|
| 파일명 지정      | 반드시 명시 필요<br>session1_cli_setup.md 파일에 ~ 추가해줘             | 생략 가능 (열려 있는 탭 기준)<br>예: 이 파일에 ~ 추가해줘         |
| 코드 범위 지정    | 함수명/클래스명 명시 또는 파일 전체 대상<br>math.py의 calculate_avg 함수 수정해줘 | 마우스 드래그 또는 커서 위치<br>이 로직 성능 최적화해줘             |
| 에러 해결       | 에러 메시지 복사/붙여넣기 필요<br>python run.py 실행 시 발생한 [에러내용] 해결해줘   | IDE 에러 로그/터미널 탭 연동<br>예: 방금 발생한 빌드 에러 고쳐줘     |
| 추천 프롬프트 스타일 | 구체적이고 독립적인 명령형<br>(작업 대상과 행위를 모두 텍스트로 완성)                 | 지시 대명사 기반의 맥락형<br>(이 부분, 여기, 방금 연 파일 등 사용 가능) |

### 3. 환경별 프롬프팅 팁 (Tip)

#### 💡 agy CLI에서 프롬프팅할 때 (명확한 정보 제공)

터미널 환경에서는 AI가 스스로 맥락을 유추하기 어려우므로, 대상을 정확하게 짚어주는 것이 중요합니다.

- 나쁜 예: "함수 에러 고쳐줘"
- 좋은 예: "src/utils.py 파일의 format\_date 함수에서 발생할 수 있는 null 에러 예외 처리를 추가해줘"

#### 💡 Antigravity IDE에서 프롬프팅할 때 (맥락 활용)

마우스 드래그와 화면 포커스를 적극 활용하여 지시 대상을 직관적으로 좁힐 수 있습니다.

- 나쁜 예: 드래그하지 않고 "여기에 코드 추가해줘" (여디가 '여기'인지 모호할 수 있음)
- "이 코드 가독성을 개선해줘" "이 부분에 예외 처리 추가해줘"

### 4. 종합 추천 (어떤 환경을 선택해야 할까?)

일반적으로 코드를 직접 작성하고 편집하는 대다수의 일상적인 개발 작업에는 **Antigravity IDE** 환경을 사용하는 것을 적극 권장합니다. 에디터가 제공하는 실시간 시각적 맥락(커서, 선택 영역, 파일 탭)이 AI의 이해도를 매우 높여주기 때문입니다. 다만, 작업의 성격에 따라 아래와 같이 두 환경을 유기적으로 교체하며 사용하는 것이 가장 좋습니다.

#### 🌟 Antigravity IDE 사용을 추천하는 상황

- **코드 편집 및 리팩토링**: 기존 함수의 구조 변경, 변수명 변경, 로직 최적화 등 코드를 직접 보며 수정할 때 (드래그 지정 가능).
- **실시간 디버깅**: 코드 실행 시 에디터나 출력 패널에 나타나는 에러 메시지를 즉시 해결하고 싶을 때.
- **UI 및 컴포넌트 개발**: HTML/CSS 구성 및 컴포넌트 연동처럼 에디터 내에서 소스 코드의 세밀한 편집이 반복될 때.

#### ⚙️ agy CLI 사용을 추천하는 상황

- **프로젝트 셋업 및 일괄 생성**: 대량의 파일 구조 생성, 프로젝트 초기화( `uv init` 등)처럼 셸 환경의 파일 제어가 주가 될 때.
- **자동화 스크립트 및 테스트 실행**: 전체 테스트 스위트 실행, 패키지 동기화( `uv sync` ), 빌드 파이프라인 점검 등 명령어 기반의 일괄 제어가 필요할 때.
- **버전 관리 및 배포**: Git 커밋, 브랜치 작업, 체크포인트 설정, 롤백 및 빌드 산출물 배포 스크립트 실행 시.
- **백그라운드 장기 실행 작업**: 대규모 리서치나 여러 에이전트를 가동해 놓고 백그라운드로 주기적으로 리포트를 받아보고 싶을 때.

### 5. 그럼에도 불구하고 CLI(Command Line Interface)가 반드시 필요한 이유

IDE가 점점 더 똑똑해지고 편리한 GUI 도구들이 많이 제공됨에도 불구하고, 개발 생태계에서 CLI가 절대 대체될 수 없으며 모든 개발자가 필수적으로 다루어야 하는 이유는 다음과 같습니다.

## ① 자동화(Automation)와 스크립팅의 기반

GUI나 IDE는 사람이 직접 마우스로 화면의 버튼을 클릭하고 입력해 주어야만 작동합니다. 반면 CLI 명령어는 단순한 텍스트 파일(스크립트) 형태로 기록하여 **사람의 개입이 전혀 없는 100% 자동화 환경**을 구축할 수 있습니다.

- 주기적인 데이터 백업, 테스트 자동 실행, 배포 자동화(CI/CD) 파이프라인 등 현대 소프트웨어 공학의 자동화 체계는 전부 CLI 명령어의 조합으로 구현됩니다.

## ② 원격 서버 및 컨테이너(Docker) 환경에서의 필수성

우리가 작성한 코드가 실제로 서비스되는 클라우드 서버(AWS, GCP 등)나 리눅스 기반 서버, 그리고 가상 격리 환경인 Docker 컨테이너 내부에는 마우스로 클릭할 수 있는 GUI 화면이나 IDE가 존재하지 않습니다.

- 오직 까만 화면(터미널)을 통해 네트워크로 접속(SSH)하는 방식만 허용되는 경우가 대부분이므로, 서버 상태를 모니터링하고 배포/운영하기 위해서는 CLI 환경에 익숙해야 합니다.

## ③ 도구 간 유기적인 연결 (파이프라인)

CLI 환경에서는 한 명령어의 실행 결과(출력)를 다른 명령어의 입력으로 즉시 연결해 주는 **파이프(Pipe, | )** 기능을 활용할 수 있습니다.

- 예를 들어, `cat logs.txt | grep "Error" | wc -l` 처럼 여러 개의 작고 강력한 도구들을 조합하여 수백만 줄의 텍스트 파일 안에서 특정 에러 라인의 개수를 단 한 줄의 명령어로 즉시 찾아낼 수 있습니다. GUI로는 이러한 유연한 연동이 매우 어렵습니다.

## ④ 압도적인 속도와 자원 효율성

무겁고 컴퓨터의 메모리(RAM)를 많이 차지하는 IDE에 비해 터미널은 리소스를 거의 차지하지 않으며 부팅과 실행 속도가 즉각적입니다. 또한 마우스를 쥐기 위해 손을 옮길 필요 없이 키보드만으로 모든 조작이 완료되므로, 명령어가 손에 익을수록 GUI 대비 작업 처리 속도가 비약적으로 빨라집니다.

## ⑤ 개발 도구들의 'CLI First' 철학

`git`, `docker`, `npm`, `uv` 등 현대 개발을 지탱하는 거의 모든 핵심 기술은 **CLI 작동을 최우선(CLI First)**으로 **상정하고 설계**됩니다. GUI 프로그램들은 이러한 내부 CLI 엔진 위에 껍데기(비주얼 인터페이스)만 씌운 것에 불과합니다. 때문에 복잡한 오류나 환경 설정의 예외가 생겼을 때는 GUI가 에러를 받고 멈춰버리기 십상이며, 결국 원인을 파악하고 강제 해결하기 위해 CLI 명령어를 사용할 수밖에 없습니다.

## ⑥ 터미널 전용 AI 에이전트 도구(예: Claude Code) 활용의 필수 조건

최근 주목을 받고 있는 에이전트형 AI 코딩 도구인 **Claude Code (클로드 코드)** 등은 자체적인 독립 그래픽 IDE(프로그램 화면)를 제공하지 않으며, **오직 터미널(CLI) 환경 내부에서 직접 실행**되도록 설계되어 있습니다. 이러한 도구를 사용해 셸(Shell) 상에서 프로젝트 분석, 자동화 테스트 실행, Git 버전 관리 등을 수행하기 위해서는 CLI 환경 구축과 셸 명령어에 대한 기본적인 이해가 필수적입니다.